Image Grids

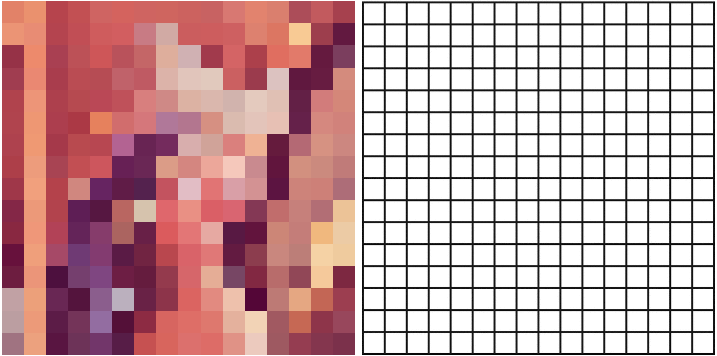
Mapping from pixel to world coordinates

Y. Huang, S. Mei, M. Thies, A. Maier | Flat-Panel CT Reconstruction, FAU Erlangen, SS 2024

How are images represented in computers?

- Pixel size: 512 * 512 * 3 (RGB channels)

Images are described by grids in computers



- Pixel size: 16 * 16 * 3 (left); 16 * 16 (right)

How to describe an image size?

- Pixel size: 512 * 512 * 3 (left); 64 * 64 * 3 (left); 64 * 64 * 3 (right)
- Also need to know the size of each pixel to describe image size (physical width and height)

How to describe an image size?

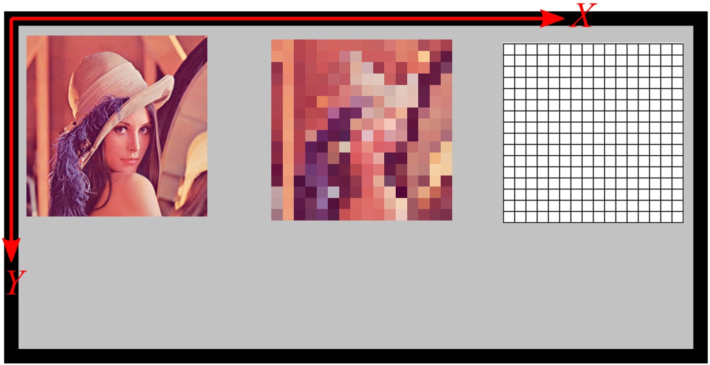
- Pixel size: 512 * 512 * 3 (RGB channels)
- If a pixel has a size of 0.5 mm (square), what is the physical size? 256 mm * 256 mm

How to describe an image size?



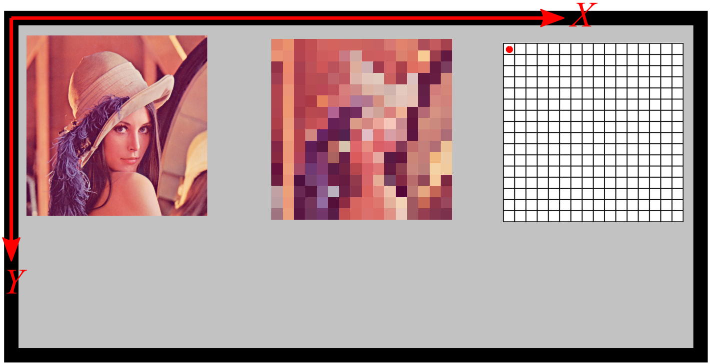
- Pixel size: 512 * 512 * 3 (left); 64 * 64 * 3 (left); 64 * 64 * 3 (right)
- Pixel sizes: 0.5 mm (left); 4 mm (middle); 0.5 mm (right)

How to describe an image position?



- Think about hanging the paint of Lenna at different positions of a wall

How to describe an image position?

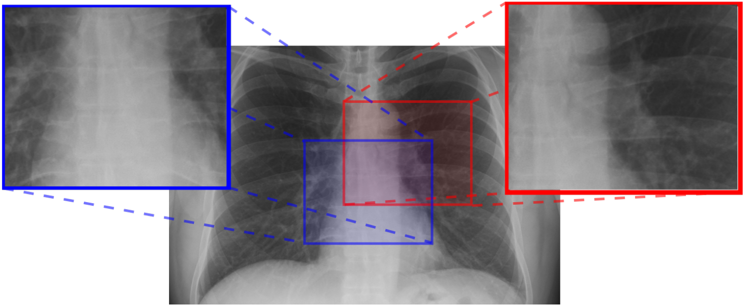


- Think about hanging the paint of Lenna at different positions of a wall
- We can use the position of the top left corner of the paint to describe its position

Pixel coordinates for X-ray images

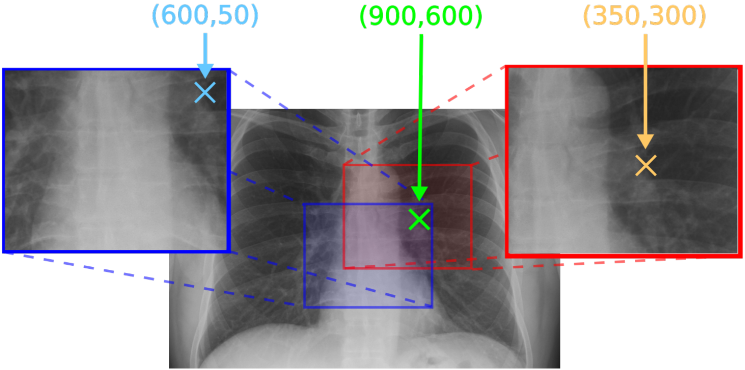
- Consider an X-ray image of arbitrary size

Pixel coordinates for X-ray images



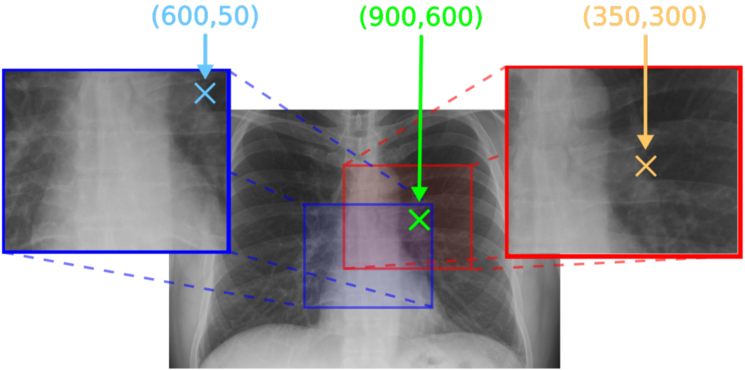
- Consider an X-ray image of arbitrary size
- ROIs drawn by Physician A and B

Pixel coordinates for X-ray images



- Consider an X-ray image of arbitrary size
- ROIs drawn by Physician A and B

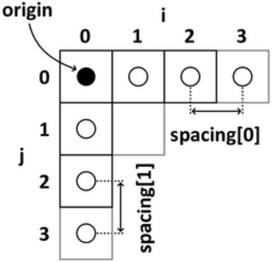
Pixel coordinates



- Pixel coordinates are all different

► **Yet, they refer to the exact same point!**

Image properties

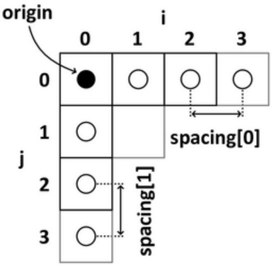


- Origin $\mathbf{o} \in \mathbb{R}^2$
- Basis vectors $\mathbf{e}_0, \mathbf{e}_1 \in \mathbb{R}^2$
- $\mathbf{e}_0^\top \cdot \mathbf{e}_1 = 0$
- $||\mathbf{e}_0|| = s_0$ is the spacing in x-direction
- $||\mathbf{e}_1|| = s_1$ is the spacing in y-direction

Image width and height:

- Width = $N_0 \times s_0$
- Height = $N_1 \times s_1$

Image properties



- Origin $\mathbf{o} \in \mathbb{R}^2$
- Basis vectors $\mathbf{e}_0, \mathbf{e}_1 \in \mathbb{R}^2$
- $\mathbf{e}_0^\top \cdot \mathbf{e}_1 = 0$
- $||\mathbf{e}_0|| = s_0$ is the spacing in x-direction
- $||\mathbf{e}_1|| = s_1$ is the spacing in y-direction

Image width and height:

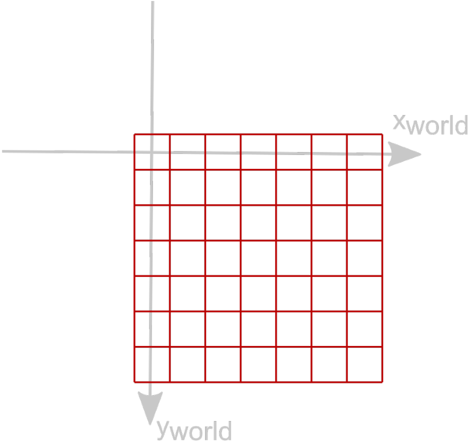
- Width = $N_0 \times s_0$
- Height = $N_1 \times s_1$

Pixel and world coordinate conversion:

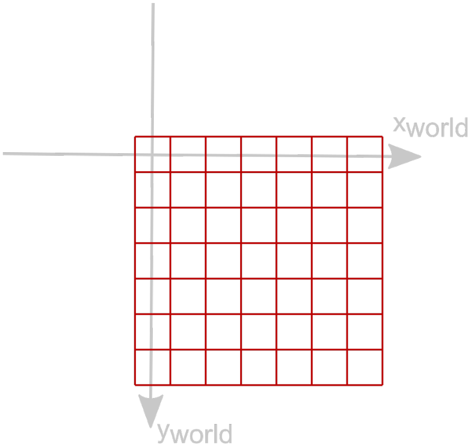
- Pixel coordinates: (i, j)
- What are its world coordinates?

How to set the pixel origin

- Set pixel origin: assign the physical/world coordinates of the top left pixel center



How to set the pixel origin



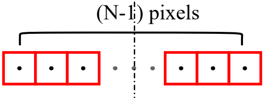
- The pixel origin and the world origin overlap

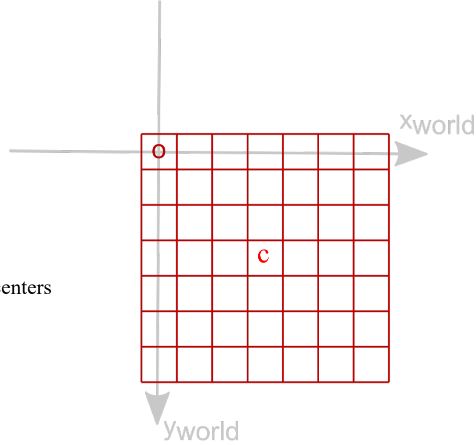
How to set the pixel origin

$o_{pixel} = [0, 0]$
$o_{world} = [o_x, o_y] = [0, 0]$

$c_{pixel} = [0.5(N_0\text{-}1), 0.5(N_1\text{-}1)]$
$c_{world}=[0.5(N_0\text{-}1)s_0, 0.5(N_1\text{-}1)s_1]$

Pixel indices stand for the pixel centers
Half a pixel shift





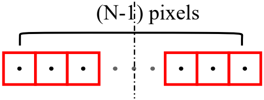
- The pixel origin and the world origin overlap

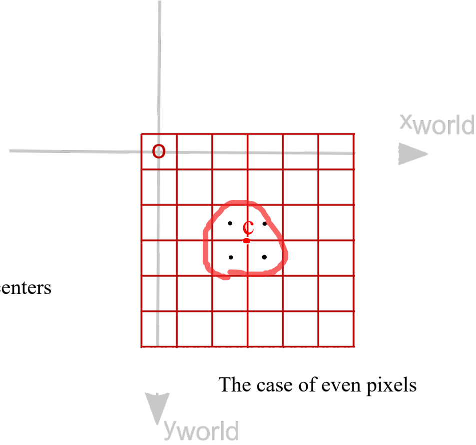
How to set the pixel origin

$o_{pixel} = [0, 0]$
$o_{world} = [o_x, o_y] = [0, 0]$

$c_{pixel} = [0.5(N_0\text{-}1), 0.5(N_1\text{-}1)]$
$c_{world}=[0.5(N_0\text{-}1)s_0, 0.5(N_1\text{-}1)s_1]$

Pixel indices stand for the pixel centers
Half a pixel shift





The case of even pixels

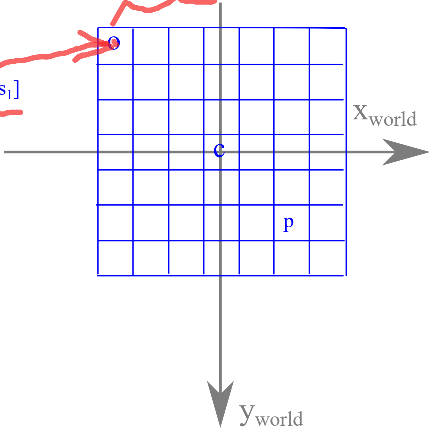
- The pixel origin and the world origin overlap

How to set the pixel origin

$o_{pixel} = [0, 0]$
$o_{world} = [o_x, o_y] = [-0.5(N_0-1)s_0, -0.5(N_1-1)s_1]$

$c_{pixel} = [0.5(N_0-1), 0.5(N_1-1)]$
$c_{world}=[0, 0]$

$p_{pixel} = [i, j]$
$p_{world} = [o_x + i*s_0, o_y + j*s_1]$



- The world origin is located at the center of the image

From image to world coordinates

Map pixel coordinates $\mathbf{p} \in \mathbb{N}^2$ to world coordinates $\mathbf{x} \in \mathbb{R}^2$

Image to world

$$\mathbf{x} = (\mathbf{e}_0\,,\, \mathbf{e}_1\,,\, \mathbf{o}) \cdot \begin{pmatrix} \mathbf{p} \\ 1 \end{pmatrix}$$

Often $\mathbf{e}_0 = \begin{pmatrix} s_0 \\ 0 \end{pmatrix}$, $\mathbf{e}_1 = \begin{pmatrix} 0 \\ s_1 \end{pmatrix}$, $\mathbf{o} = \begin{pmatrix} -0.5 \cdot (N_0 - 1.0)s_0 \\ -0.5 \cdot (N_1 - 1.0)s_1 \end{pmatrix}$,

where $\mathbf{N} \in \mathbb{N}^2$ is the image dimension (number of pixels).

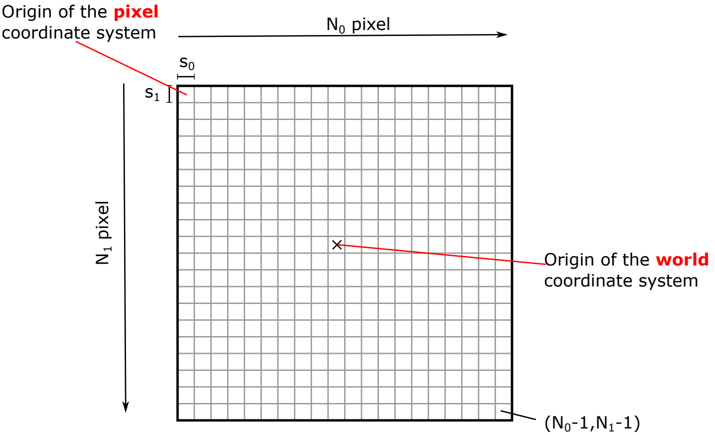
From world to image coordinates

If $\mathbf{e}_0 = \begin{pmatrix} s_0 \\ 0 \end{pmatrix}$, $\mathbf{e}_1 = \begin{pmatrix} 0 \\ s_1 \end{pmatrix}$ the inversion is easy:

World to image

$$\mathbf{p} = \begin{pmatrix} 1/s_0 & 0 \\ 0 & 1/s_1 \end{pmatrix} \cdot (\mathbf{x} - \mathbf{o})$$

Overview


Origin of the **pixel** coordinate system
N_0 pixel
s_0
s_1
N_1 pixel
Origin of the **world** coordinate system
$(N_0\text{-}1,N_1\text{-}1)$

Warning Warning Warning Warning Warning

80% of the "hard to find mistakes" in the end are due to ignoring the correct handling of the two coordinate systems.
In your own interest, please consider the following advice:

- Do not test with a spacing of 1.0 only.
- Set the origin and spacing correctly when initializing your Grid.
- Do not forget the half-pixel shift.
- Consistency: numpy 0-th dimension for height, 1st dimension for width.
- Once correctly set, use the member functions of your Grid to convert from the two coordinate systems into each other: **index_to_physical**(...) and **physical_to_index**(...).